

符号计算入门

作者： R. De Graeve, B. Parisse, B. Ycart

机构： 格勒诺布尔约瑟夫·傅里叶大学 (Universit  Joseph Fourier,
Grenoble)

译者： Zongwen Chi (结合gemini 3.6 flash扩展模式)

Xcas 是一款开源的计算机代数系统 (CAS/符号计算软件)。

下载地址为： http://www-fourier.ujf-grenoble.fr/~parisse/giac_fr.html

它是与 Maple 和 MuPAD 相当的软件，并且与它们高度兼容。

可以对 Xcas 进行配置，使其能够接受 Maple、MuPAD 或 TI 计算器

(TI-89、Voyage 200、Nspire CAS) 的语法。本文将仅限于介绍 Xcas 本身的语法。

本入门教程旨在帮助具备一定数学基础 (相当于高中理科或大学理工科一年级水平) 并具备基本计算机操作经验的用户快速上手 Xcas。

本文并不会展示 Xcas 的所有功能。特别地，我们不会涉及交互式几何、Logo 海龟绘图以及电子表格功能。如需进一步的高级操作，请参考软件主页提供的在线帮助以及各类相关文档。

接下来的内容旨在通过介绍一些最常用的命令来帮助初学者。

建议在启动 Xcas 后阅读本文 (

在 Windows 下，点击快捷方式 xcasfr.bat；

在 Linux Gnome 下，点击 Education 菜单中的 Xcas，或在 Terminal 中输入 xcas & 并按下 Enter 键；

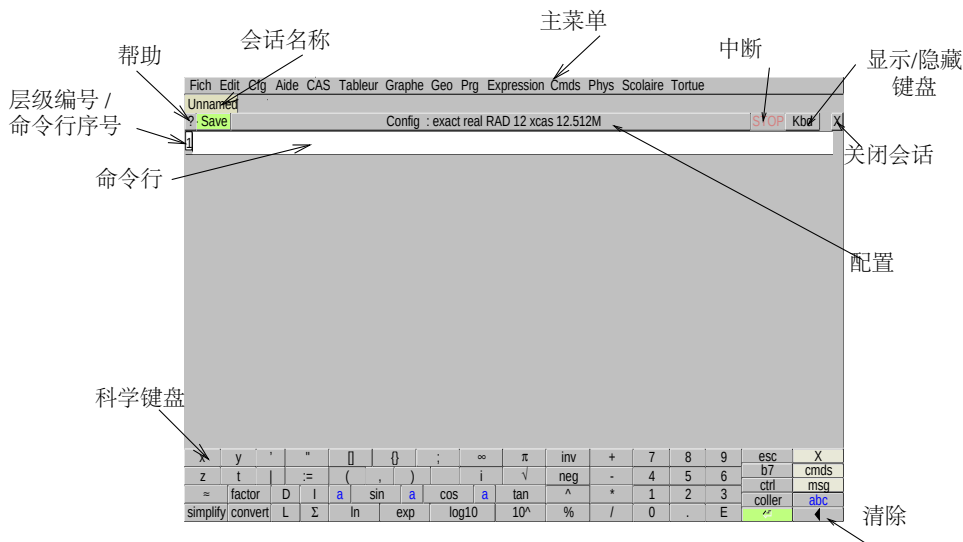
在 Mac 下，点击 Applications 菜单中的 Xcas)，并逐一执行文中给出的示例命令，以理解其作用与效果。

目录与索引存放在本文末尾，参见附录 A。

1 新手入门

1.1 界面

目前，您只需输入您的第一批命令。界面还提供了许多其他功能，您将在后续内容中逐步了解。启动 Xcas 时，界面如下图所示：



您可以调整其大小。自上而下，该界面依次包含：

- 一个可点击的菜单栏：Fich, Edit, Cfg, Aide, CAS, Expression, Cmd, Prg, Graphic, Geo, Tableur, 等。
- 一个显示会话名称的标签页，若会话尚未保存则显示为 Unnamed（可以并行打开多个会话，从而拥有代表各个会话的多个标签页），
- 一个会话管理区域，包含：
 - 一个用于打开帮助索引的 ? 按钮，
 - 一个用于保存会话的 Save 按钮，
 - 一个显示 CAS 配置 (Config: exact real ...) 并允许对其进行修改的按钮，
 - 一个红色的 STOP 按钮，用于中断耗时过长的计算，
 - 一个 Kbd 按钮，用于显示类似于计算器的虚拟键盘（如上图所示）。它可以方便您的输入，并可通过 Kbd->msg (或通过菜单 Cfg->Montrer->msg) 显示消息窗口，以及通过 Kbd->cmds (或通过菜单 Cfg->Montrer->bandeau) 显示命令面板，
 - 一个 x 按钮，用于关闭当前会话，
- 一个编号为 1 的白色矩形区域（第一条命令行），您可以在其中输入您的第一条命令（如有必要，请点击该命令行使光标出现）：1+1，随后按下“回车”键（依据键盘不同，标注为 Enter 或 Return）。计算结果将显示在下方，同时会自动打开一条编号为 2 的新命令行。您可以保存所作的修改。

您可以修改界面的外观，并保存您的修改以供日后使用（菜单 Cfg）。

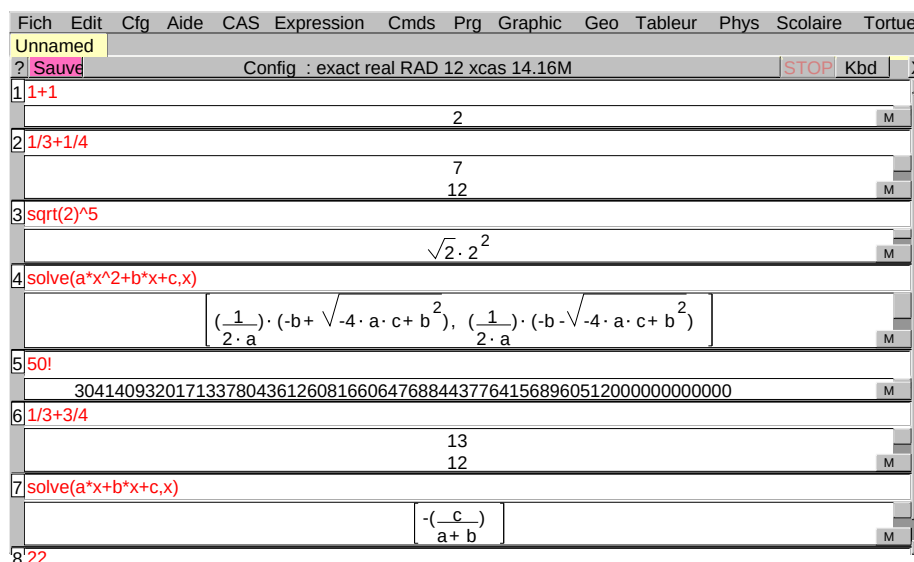
目前，您只需在接下来的各个命令行中依次输入命令。如果您使用的是本教程的 HTML 版本，可以直接从浏览器中复制并粘贴文中给出的命令。每输入一条命令行，按下“回车”键（Enter）即可执行。例如，您可以尝试执行以下命令：

```
1/3+1/4
sqrt(2)^5
solve(a*x^2+b*x+c,x)
50!
```

所有命令都会保存在内存中。因此，您可以使用 Ctrl+↑ 组合键翻阅会话的历史记录，重新调出之前的命令并进行修改。例如，您可以尝试修改前面的命令并执行以下操作：

```
1/3+3/4
solve(a*x+b*x+c,x)
```

于是得到：



此时，右侧会出现一个滚动条，用于在会话的各个层级之间浏览，例如在此处可以查看第 8 个层级。 **Edit**（编辑）菜单允许您构建比单纯的命令序列更复杂的会话。您可以创建命令行分组（章节）、添加注释，或者将多个层级合并为一个层级。

1.2 命令与在线帮助

命令按主题分类列于顶部栏的菜单中: **CAS**, **Expression**, **Cmds**, **Prg**, **Graphic**, **Geo**, **Tableur**, **Phys**, **Scolaire**, **Tortue**. 某些菜单被称为“助手”（Assistant）菜单，这是因为其中命令按主题进行了分类并附有详细说明（如 **CAS** 菜单），或者因为在选择命令后，会弹出一个对话框要求您具体指定所选命令的参数（如 **Tableur** ► **Maths** 菜单或 **Graphic** 菜单）。

其他菜单包含了各种命令的名称： **Cmds** 菜单包含所有符号计算（calcul formel）命令， **Prg** 菜单包含编程中使用的所有命令， **Geo** 菜单包含所有几何命令……当您从菜单中选择一条命令时，

- 要么弹出一个对话框，允许您指定命令的参数（例如通过 **Graphic** 菜单绘制曲线，或通过 **Tableur** ► **Maths** 菜单进行统计分析）；
- 要么将该命令直接复制到命令行中。要了解该命令的语法，可以点击左上角的 ? 按钮，或显示消息区域（通过菜单 **Cfg**->**Montrer**->**msg**）。您还可以：
 - 打开所选命令的帮助索引（如果在通用配置菜单 **Cfg**->**Configuration generale** 中勾选了 **Aide index automatique** 复选框，该操作会自动执行）。此时需点击 **OK** 按钮将命令复制到命令行中（前提是光标位于某条命令行内）。您也可以点击 **Details** 按钮，在浏览器中查看该命令对应的详细手册页面。
 - 在通用配置菜单（**Cfg**-> **Configuration generale**）中勾选 **Aide HTML automatique** 复选框，在浏览器中自动打开手册的对应页面。

Aide（帮助）菜单包含各种可能的帮助形式：用户指南（界面）、参考指南（**Manuels**->**Calcul formel**，提供每条命令的详细帮助）、**Index**（按字母顺序排列的命令列表，带有方便快速定位的输入框）以及关键词搜索。

如果您已经知道某条命令的名称并希望检查其语法（例如 **factor**），您可以输入命令名称的开头（例如 **fact**），然后按下 **Tab** 键（在法语键盘上位于 **A** 键左侧）或点击左上角的 ? 按钮。

此时，命令索引窗口将会弹出，自动定位到第一个可能的补全项，并显示每条命令的简要帮助信息。例如，如果您想因式分解一个多项式，猜想命令名称以 **fact** 开头，只需输入 **fact** 并按下 **Tab** 键，用鼠标选中 **factor**（或其中的一个示例），最后点击 **OK** 即可。

您也可以输入 **?factor** 来直接获取简要帮助。如果您输入的名称无法被识别，系统会为您推荐相近的命令。

1.3 输入命令

按“回车”键 (Enter) 即可直接执行一行命令。如果不希望显示运行结果，可以在命令行末尾加上 `;;`，然后再按“回车”键确认。也可以在执行前连续编写多条命令，只要用分号 (`;`) 将它们隔开即可。按“回车”键 (Enter) 即可直接执行一行命令。如果不希望显示运行结果，可以在命令行末尾加上 `;;`，然后再按“回车”键确认。也可以在执行前连续编写多条命令，只要用分号 (`;`) 将它们隔开即可。

刚开始使用时，许多错误都源于对数学表达式的格式转换不当：`Xcas` 无法直接理解您按日常习惯写出的数学式。虽然您可以通过键盘输入 $ax^2 + bx + c$ ，但计算机无法自动理解您是想对 x 求平方、将其与 a 相乘等操作。您必须明确指定每一个运算符，正确的语法应为 $a*x^2+b*x+c$ 。在输入命令时，乘法必须用星号 (`*`) 表示，而在系统返回的答案中，乘法则会显示为一个点号 (`.`)。我们需要特别强调的是：对于 `Xcas` 而言， ax 是一个由两个字母构成的单一变量名，而非 a 与 x 的乘积。

运算	
<code>+</code>	加法
<code>-</code>	减法
<code>*</code>	乘法
<code>/</code>	除法
<code>^</code>	乘方 / 幂运算

只要注意若干细节，表达式的书写就相当直观。圆括号的作用与通常一致，用于指定运算顺序；方括号则专门用于列表和索引（下标）。运算符的优先级遵循标准规则（乘法优先于加法，乘方优先于乘法）。例如：

- $2*a+b$ 会返回 $2 \cdot a + b$
- $a/2*b$ 会返回 $\frac{a \cdot b}{2}$
- $a/2/b$ 会返回 $\frac{a}{2 \cdot b}$
- `normal(a/2/b)` 会返回 $\frac{a}{2 \cdot b}$
- a^2*b 会返回 $a^2 \cdot b$

如有疑问，加上括号总是更稳妥的做法，以确保运算顺序符合预期。

命令和对应的结果都是带有编号的。不过，如果您修改了某一条命令行，它仍会保留原有的编号。您可以通过 `ans()`（即 `answer`）来调取上一条结果，也就是最后一次执行/求值的命令所返回的答案。

2 符号计算的对象

2.1 数值

数分为精确数和近似数。精确数包括预定义常数、整数、整数分数（有理数），以及更广义地，任何仅包含整数和常数的表达式，例如 $\sqrt{2} * e^{(i * \pi / 3)}$ 。近似数采用标准的科学记数法表示：整数部分、小数点和小数部分（末尾可接字母 e 及指数）。例如， 2 是一个精确整数， 2.0 是该整数的近似版本； $1/2$ 是一个有理数， 0.5 是该有理数的近似版本。`Xcas` 可以处理任意精度的整数：您可以尝试输入 $500!$ ，数一数返回结果共有多少位数字。

通过 `evalf` 命令可以将精确值转换为近似值；通过 `exact` 命令可以将近似值转换为精确的有理数。如果参与计算的所有数字都是精确数，计算将在精确模式下进行；只要其中有一个数字是近似数，计算就会在近似模式下进行。因此，`1.5+1` 会返回一个近似数，而 `3/2+1` 则会返回一个精确数。

```
sqrt(2)                                译注：根号2的精确值
evalf(sqrt(2))                        译注：根号2的近似值
sqrt(2)-evalf(sqrt(2))
exact(evalf(sqrt(2)))*10^9            译注：先计算根号2的浮点近似值，再将其转回精确的分数形式，最后放大 10^9倍
exact(evalf(sqrt(2)*10^9))            (即乘以10亿)。
```

对于近似实数，默认精度接近 14 位有效数字（根据 `Xcas` 版本的不同，规格化浮点实数的相对精度为 53 或 45 比特）。您可以通过将所需的小数位数作为 `evalf` 的第二个参数来提高精度。

```
evalf(sqrt(2),50)
evalf(pi,100)
```

此外，您也可以通过修改变量 `Digits` 来更改所有计算的默认精度。

```
Digits:=50
evalf(pi)
evalf(exp(pi*sqrt(163)))
```

字母 `i` 专门保留用于表示虚单位（根号-1），不能被重新赋值；特别是，不能将其用作循环索引（循环变量）。

```
(1+2*i)^2
(1+2*i)/(1-2*i)
e^(i*pi/3)
```

`Xcas` 区分无符号无穷大 `infinity` (∞)，正无穷大 `+infinity` 或 `inf(+∞)`，以及负无穷大 `-infinity` 或 `-inf(-∞)`。

```
1/0; (1/0)^2; -(1/0)^2
```

预定义常数	
<code>pi</code>	$\pi \simeq 3.14159265359$
<code>e</code>	$e \simeq 2.71828182846$
<code>i</code>	$i = \sqrt{-1}$
<code>infinity</code>	∞
<code>+infinity</code> ou <code>inf</code>	$+\infty$
<code>-infinity</code> ou <code>-inf</code>	$-\infty$

2.2 字符与字符串

字符串用双引号 (") 括起来。字符则是仅包含一个元素的字符串。

```
s:="azertyuiop"  
size(s)  
s[0]+s[3]+s[size(s)-1]  
concat(s[0],concat(s[3],s[size(s)-1]))  
head(s)  
tail(s)  
mid(s,3,2)  
l:=asc(s)  
ss:=char(l)  
string(123)  
expr(123)  
expr(0123)
```

字符串	
asc	字符串->ASCII 码列表
char	ASCII码列表->字符串
size	字符数 / 字符串长度
concat 或 +	字符串拼接
mid	截取子字符串 / 提取片段
head	首字符
tail	去除首字符后的剩余字符串
string	数字或表达式->字符串
expr	字符串->数字（十进制或八进制）或表达式

2.3 变量

如果一个变量未包含任何值，我们就称其为形式变量（符号变量）：在被赋值之前，所有变量默认都是形式变量。赋值操作 `:=` 表示。在会话开始时，`a` 是一个形式变量；但在执行指令 `a:=3`，之后，`a` 就变成了已赋值变量，随后的所有计算中 `a` 都会被替换为 `3`，输入 `a+1` 将返回 `4`。会保存您当前会话中的所有内容。如果希望变量 `a` 在赋值后重新变回形式变量，必须通过 `purge(a)`。将其“清空/清除”

切勿混淆：

- 符号 `:=` 表示赋值操作
- 符号 `==` 表示布尔/逻辑相等（这是一个二元运算符，返回 `1` 或 `0`；其中 `1` 代表 `true` 即“真”，`0` 代表 `false` 即“假”）。
- 符号 `=` 用于定义方程

```

a==b
a:=b
a==b
solve(a=b,a)
solve(2*a=b+1,a)

```

可以使用 `assume` 命令对变量添加某种假设，例如 `assume(a>2)`。假设是一种特殊形式的赋值，它会清除该变量此前可能已被赋予的值。在计算过程中，变量不会被直接替换为数值，但系统会尽可能地利用该假设。例如，在设置了假设 `a>2` 后，输入 `abs(a)` 将会直接返回 `a`。

```

sqrt(a^2)
assume(a<0)
sqrt(a^2)
assume(n,integer)
sin(n*pi)

```

`subst` 函数允许在不影响变量本身（即不对其进行赋值）的前提下，将表达式中的某个变量替换为一个数值或另一个表达式。

```

subst(a^2+1,a=1)
subst(a^2+1,a=sqrt(b-1))
a^2+1

```

注意：若要通过引用方式将值存入变量（例如修改列表、向量或矩阵中的某个元素），并在不重新创建新列表的前提下原地修改（**in-place**）现有列表，应使用指令 `=<` 来代替 `:=`。该指令比 `:=` 更高效，因为它省去了复制整个列表的时间开销。

2.4 表达式

表达式是由数字和变量通过运算符组合而成的结构，例如 x^2+2x+c 。

当确认/执行一条命令时，`Xcas` 会将变量替换为其对应的值（如果该变量已被赋值），并执行相应的计算操作。

```

(a-2)*x^2+a*x+1
a:=2
(a-2)*x^2+a*x+1

```

在求值（计算）过程中，某些化简操作会自动执行：

- 整数与分数的运算，包括将其约分为最简分数（既约分数）形式。
- 平凡的化简（基础化简），例如 $x+0=x$ 、 $x-x=0$ 、 $x*1=x$ 等。
- 一些三角函数化简，例如 $\cos(-x)=\cos(x)$ 、 $\cos(\pi/4)=(1/2)^{(1/2)}$ 、 $\tan(\pi/4)=1$ 等。

我们将在下一节中了解如何进行更多的化简。

2.5 展开与化简

除上一节所述的规则外，系统不会进行自动的化简。这主要有两个原因：
计算开销：非平凡（复杂）的化简有时非常耗时，因此是否执行化简完全由用户决定；
多解性：根据具体的使用目的，同一个表达式通常存在多种不同的化简形式。用于转换表达式的主要命令如下：

- `expand`：展开表达式，仅考虑乘法对加法的分配律以及整数幂的展开。
- `normal` 与 `ratnormal`：在执行效率与化简效果之间具备良好的平衡，它们将有理分式（两个多项式的比值）写成已展开的最简分式形式；`normal` 会考虑代数数（例如 `sqrt(2)`），而 `ratnormal` 则不会。两者均不考虑超越函数之间的关系（例如 `sin` 与 `cos`）。
- `factor`：比前面的函数稍慢，它将分式写成因式分解后的最简形式。
- `simplify`：尝试在应用 `normal` 之前先将表达式归结为代数独立的变量。这种方法时间开销更大且过程“不可见”（无法控制中间的重写步骤）。涉及代数扩张（例如平方根）的化简，有时需要调用两次该函数，和/或配合假设（`assume`）消除绝对值后，才能得到理想的化简结果。
- `tsimplify`：尝试归结为代数独立的变量，但随后不会应用 `normal`。

在顶部菜单栏的 `Expression`（表达式）菜单中，各个子菜单均属于重写（改写）菜单，其中包含了更多用于进行特定或专用转换的函数。

```
b:=sqrt(1-a^2)/sqrt(1-a)
ratnormal(b)
normal(b)
tsimplify(b)
simplify(b)
simplify(simplify(b))
assume(a<1)
simplify(b)
simplify(simplify(b))
```

`convert` 函数允许将一个表达式转换为另一个等价的形式，具体的转换格式由第二个参数指定。

```
convert(exp(i*x), sincos)
convert(1/(x^4-1), partfrac)
convert(series(sin(x), x=0, 6), polynom)
```


变换	
<code>simplify</code>	化简:深度化简
<code>tsimplify</code>	化简 (较弱版本): 较轻量的化简
<code>normal</code>	标准型 / 规范形式: 化简为展开的最简分式, 支持代数数
<code>ratnormal</code>	标准型 (较弱版本): 针对纯有理式的规范化
<code>expand</code>	展开: 展开乘法与整数幂
<code>factor</code>	因式分解: 对表达式进行因式分解
<code>assume</code>	添加假设: 为变量设定条件约束
<code>convert</code>	转换为指定格式: 按指定格式重写表达式

2.6 函数

Xcas中预定义了许多函数, 特别是各种常见的初等/经典函数。最常用的函数如下表所示; 对于其他函数, 可参见菜单 `Cmds->Reel->Special`。

此外, 用户也可以自定义函数, 例如:

– 单变量函数的定义:

```
f(x) := x*exp(x) ou
f := x->x*exp(x) ou
f := unapply(x*exp(x), x)
```

– 导函数的定义: :

```
g := function_diff(f) ou
g := unapply(diff(f(x), x), x)
```

注意 `g(x) := diff(f(x), x)` 是无效的! 因为 `:=` 右侧的表达式在定义函数时不会被立即求值/评估.....这种情况下必须使用 `unapply`。

– 二元函数的定义:

```
h(r, t) := (r*exp(t), r*t) 或
h := (r, t) -> (r*exp(t), r*t);
```

– 由二元函数构造一个“将单变量映射为另一个函数”的函数:

```
k(t) := unapply(h(r, t), r)
```

注意: 这里的 `k(t)` 本身是一个关于变量 `r` 的函数, 它将 `r` 映射为 `h(r, t)`。例如: `k(1)(2) = (2*exp(1), 2)`。在这种情况下, 同样必须使用 `unapply`。

常见函数	
abs	绝对值
sign	符号函数 (-1,0,+1)
max	最大值
min	最小值
round	四舍五入 / 舍入
floor	向下取整 / 整数部分 (<= 该数值的最大整数)
frac	小数部分
ceil	向上取整 (>= 该数值的最小整数)
re	实部
im	虚部
abs	模 / 复数模长
arg	幅角
conj	共轭复数
affixe	复数坐标 / 复点
coordonees	坐标
factorial 或 !	阶乘
sqrt	平方根
exp	指数函数
log	自然对数
ln	自然对数
log10	常用对数 / 以 10 为底的对数
sin	正弦
cos	余弦
tan	正切
cot	余切
asin	反正弦
acos	反余弦
atan	反正切
sinh	双曲正弦
cosh	双曲余弦
tanh	双曲正切
asinh	反双曲正弦
acosh	反双曲余弦
atanh	反双曲正切

要创建一个新函数，需要借助包含该变量的表达式来进行声明。例如，表达式 $x^2 - 1$ 的定义形式为 $x^2 - 1$ 。要将其转换为将 x 映射为 $x^2 - 1$ 的函数 f ，存在以下三种方法：

```
f(x) := x^2-1
f:=x->x^2-1
f:=unapply(x^2-1,x)
```

```
f(2);
f(a^2);
```

如果 f 是单变量函数, 且 E 是一个表达式, 则 $f(E)$ 是另一个表达式。切记不可混淆函数与表达式。

如果定义 $E := x^2 - 1$, 那么变量 E 中保存的就是表达式 $x^2 - 1$. 若要获取该表达式在 $x = 2$ 时的值, 必须写作 `subst(E, x=2)`, 而不能写作 $E(2)$, 因为 E 并不是一个函数。

在定义函数时, 赋值符号右侧的内容不会被立即求值/评估。因此写作

```
E:=x^2-1;
```

$f(x) := E$ 定义的实际上是函数 $f: x \mapsto E$ (因为 E 本身未被展开求值);

相比之下, 写作 $E := x^2 - 1; f := \text{unapply}(E, x)$ 则能正确定义函数 $f: x \mapsto x^2 - 1$ (因为 E 在此时被求值并替换为了对应的表达式)。

函数之间可以进行加法或乘法运算, 例如: $f := \sin * \exp$. 要对函数进行复合, 可以使用 `@` 运算符; 若要将一个函数与自身连续复合多次, 则使用 `@@` 运算符。

```
f:=x->x^2-1;
(f@f)(2);
(f@sqrt)(a);
f1:=f@sin
f2:=f@f
f3:=f@@3
f1(a)
f2(a)
f3(a)
```

可以定义取值为 $\mathbb{R}$ 的多元函数, 例如:

```
f(x,y):=x+2*y
```

以及取值为 $\mathbb{R}^p$ 的多元函数, 例如:

```
f(x,y):=(x+2*y,x-y)
```

2.7 列表、序列与集合

`xcas` 区分了多种由逗号分隔的对象集合类型:

- 列表 (位于方括号内)
- 序列 (位于圆括号内)
- 集合 (位于百分号-大括号内, 如 `%{ % }`)

```
liste:=[1,2,4,2]
sequence:=(1,2,4,2)
ensemble:=%{1,2,4,2%}
```

列表可以嵌套列表 (矩阵即属于此类), 而序列则是扁平的 (序列中的元素不能再是序列)。在集合中, 元素的顺序并不重要, 且每个元素都是唯一的。此外还存在另一种称为表 (table) 的结构, 我们将在后文中另作介绍。

只需将序列括在方括号内即可将其转换为列表, 或括在带 `%` 前缀的大括号内即可将其转换为集合。使用 `op` 可以将列表转换为其对应的序列; 而将序列括在方括号内或使用 `nop` 函数, 则将其转换为对应的列表。列表的元素数量可以通过 `size` (或 `nops`) 函数获取。

```

se:=(1,2,4,2)
li:=[se]
op(li)
nop(se)
nops(se)
%{se%}
size([se])
size(%{se%})

```

要创建列表或序列，可以使用迭代命令，例如 `$` 或 `seq`（对表达式进行迭代），或者使用 `makelist`（借助函数来定义列表）。

```

1$5
k^2 $ (k=-2..2)
seq(k^2,k=-2..2)
seq(k^2,k,-2..2)
[k^2$(k=-2..2)]
seq(k^2,k,-2,2)
seq(k^2,k,-2,2,2)
makelist(x->x^2,-2,2)
seq(k^2,k,-2,2,2)
makelist(x->x^2,-2,2,2)

```

空序列记为 `NULL`，空列表记为 `[]`。添加元素：要向序列中添加元素，只需用逗号将序列与新元素分隔开即可；要向列表中添加元素，则使用 `append` 函数。

元素访问：可以通过方括号内的索引来访问列表或序列中的元素，其中第一个元素的索引为 0。

```

se:=NULL; se:=se,k^2$(k=-2..2); se:=se,1
li:=[1,2]; (li:=append(li,k^2))$(k=-2..2)
li[0],li[1],li[2]

```

多项式通常由表达式定义，但也可以表示为其系数按降幂（次数降序）排列的列表，界定符为 `poly1[` 和 `]`。同时还存在多元多项式的表示方法。函数 `symb2poly` 与 `poly2symb` 于在“表达式表示法”与“列表表示法”之间进行相互转换。其第二个参数用于指定处理的是一元多项式（传入变量名）还是多元多项式（传入变量列表）。

序列与列表	
<code>E\$(k=n..m)</code>	创建序列
<code>seq(E,k=n..m)</code>	创建序列
<code>[E\$(k=n..m)]</code>	创建列表
<code>makelist(f,k,n,m,p)</code>	创建列表
<code>op(li)</code>	从列表转换为序列
<code>nop(se)</code>	从序列转换为列表
<code>nops(li)</code>	元素数量 / 元素个数
<code>size(li)</code>	元素数量 / 元素个数
<code>sum</code>	元素求和
<code>product</code>	元素求积
<code>cumSum</code>	累加和
<code>apply(f,li)</code>	对列表元素应用函数
<code>map(li,f)</code>	对列表元素应用函数
<code>map(li,f,matrix)</code>	对矩阵元素应用函数
<code>poly2symb</code>	由列表构造多项式
<code>symb2poly</code>	提取多项式系数

2.8 计算时间与内存占用

符号计算（计算机代数）面临的主要问题是中间计算的复杂性。这种复杂性既体现在执行命令所需的时间上，也体现在所需的内存空间上。**xcas** 函数中内置的算法非常高效，但它们无法保证在所有情况下都是最优解。

获取执行时间：函数 `time` 可以用来查看某条命令的执行时间（如果执行时间极短，**xcas** 会多次重复运行该命令以返回更精确的时间测量）。

查看内存使用：在 **Unix** 版本中，所使用的内存大小会显示在状态栏（按钮栏）中。

如果某条命令的运行时间超过了数秒，很有可能是发生了输入错误。请毫不犹豫地中断执行（点击右上角红色的 **STOP** 按钮；在此之前，建议先对当前会话进行保存）。

3 分析工具

3.1 导数

函数 `diff` 用于计算表达式关于其一个或多个变量的导数。

要对函数 f 求导，可以对表达式 $f(x)$ 调用 `diff`，但这样得到的结果是一个表达式。如果希望直接定义导函数，则需要使用 `function_diff`。

```
E:=x^2-1; diff(E);
f:=unapply(E,x); diff(f(x));
f1:=function_diff(f);f1(x);
```

不应该通过 `f1(x):=diff(f(x))` 的方式来定义导函数，因为在这样的定义中，`x` 会同时具有两个互不兼容的含义：一方面它是求导的形式变量，另一方面它又是函数 `f1` 的自变量。

另一方面，这种定义方式会在每次调用函数时都重新对 `diff` 进行一次求值（因为赋值语句右侧的内容默认不会被立即求值），这会导致运行效率非常低下。

应当使用 `f1:=function_diff(f)`，或者

`f1:=unapply(diff(f(x)),x)`。

函数 `diff` 适用于任意变量的组合，能够用于计算连续的偏导数（高阶偏导数）。

```
E:=sin(x*y)
diff(E,x)
diff(E,y)
diff(E,x,y)-diff(E,y,x)
simplify(ans())
diff(E,x$2,y$3)
```

如果 `diff` 的第二个参数是一个列表，则会返回一个由各导数组成的列表。例如，要计算 $\sin(xy)$ 的梯度：

```
diff(sin(x*y),[x,y]) //计算 sin(x*y) 关于 x 和 y 的偏导数列表（即梯度）
```

（也可以直接使用 `grad` 命令）。

此外，还有一些特定命令可用于计算偏导数的经典（算符）组合。

导数	
<code>diff(ex,t)</code>	表达式关于 t 的导数
<code>function_diff(f)</code>	函数的导函数
<code>diff(ex,x\$n,y\$m)</code>	偏导数
<code>grad</code>	梯度
<code>divergence</code>	散度
<code>curl</code>	旋度
<code>laplacian</code>	拉普拉斯算子
<code>hessian</code>	黑塞矩阵

3.2 极限与泰勒展开

函数 `limit` 用于计算有限极限或无穷极限（在极限存在的情况下）。单侧极限：可以通过传入第四个参数（+1 或 -1）来求解右极限或左极限。

参数假设：当函数依赖于某个参数时，计算出的极限结果可能取决于使用 `assume` 函数对该参数所设定的假设条件。

```
limit(1/x,x,0)
limit(1/x,x,0,1)
limit(1/x,x,0,-1)
limit(a/x,x,0,1)
assume(a>0)
limit(a/x,x,0,1)
```

对于泰勒展开，有两个函数可用：`series` 和 `taylor`。

两者的区别在于：使用 `series` 时必须显式指定展开的阶数，而 `taylor` 的默认展开阶数为 6。

用户请求的泰勒展开阶数由 `xcas` 在内部用于进行展开运算。若计算过程中发生了抵消与化简，最终获得的展开阶数可能会低于预期，此时需要传入更大的阶数重新进行计算。

返回的表达式由泰勒多项式和余项组成。余项中会出现一个 `order_size` 函数，该函数满足：对任意 $a > 0$ ，当 $x \rightarrow 0$ 时， $x^a \text{order_size}(x) \rightarrow 0$ 。若要去除余项并仅保留泰勒多项式，可以使用带有 `polynom` 选项的 `convert` 函数。

```
taylor(1/(x^2+1),0)
taylor(1/(x^2+a^2),x=0)
series(1/(x^2+1),0,11)
series(1/(x^2+1),+infinity,11)
series(tan(x),pi/4,3)
series(sin(x)^3/((1-cos(x))*tan(x)),0,4)
series(sin(x)^3/((1-cos(x))*tan(x)),0,6)
series(tan(sin(x))-sin(tan(x)),0,13)
convert(ans(),polynom)
series(f(x),0,3)
g:=f@f; series(g(x),0,2)
```

极限与泰勒展开	
<code>limit(ex,x,a)</code>	在a处的极限
<code>limit(ex,x,a,1)</code>	在a处的右极限
<code>limit(ex,x,a,-1)</code>	在a处的左极限
<code>taylor(ex,a)</code>	在a处的6阶泰勒展开
<code>series(ex,a,n)</code>	在a处的n阶泰勒展开

3.3 原函数与积分

函数 `int` 用于计算表达式关于 x 或指定变量的原函数（不定积分）。

指定积分变量：如果表达式中包含多个变量，需要明确指定对哪一个变量进行积分。

定积分计算：在积分变量之后追加两个参数 a 和 b ，即可计算在区间 $[a,b]$ 上的定积分。

```
int(x^2-1)
int(x^2-1,x,-1,1)
int(x*y,x)
int(x*y,y,0,x)
int(int(x*y,y,0,x),x,0,1)
```

为了计算积分，符号计算软件（计算机代数系统）会先寻找原函数，随后在积分上下限之间对其求值，以获得精确值。

但在某些情况下，寻找原函数是没有必要的：要么是因为不存在能够用初等函数表示的原函数，要么是因为数值计算更加适用（例如

原函数的计算时间过长、函数在积分区间内存在奇点等)。

在这种情况下, 可以通过使用 `evalf` 来求解近似值 (数值解), 或者直接使用由 `evalf` 调用的 `romberg` 函数。

```
int(exp(-x^2))
int(exp(-x^2), x, 0, 10)
evalf(int(exp(-x^2), x, 0, 10))
romberg(exp(-x^2), x, 0, 10)
ans()/sqrt(pi)
```

积分	
<code>int(E)</code>	表达式的原函数
<code>int(E, x, a, b)</code>	精确积分
<code>romberg(E, x, a, b)</code>	近似积分 / 数值积分

3.4 方程求解

与积分计算类似, 方程求解也分为以下两种方式:

- 精确求解: 在可能的情况下返回方程的所有精确解 (例如对于某些多项式方程, 或可以化归为多项式方程的情形)。
- 近似求解: 通过迭代算法计算出其中某一个解的数值近似值。

精确求解通过 `solve` 函数实现, 其第一个参数为一个方程。如果未显式指定方程的右端项, 系统默认右端项为 0。默认情况下, `solve` 不会返回复数解。若要获取复数解, 需要在 CAS 配置中勾选 `Complex` (操作路径: `Cfg -> CAS Configuration`, 或点击状态栏/按钮栏上的 `Config : exact...`)。请在激活 `Complex` 选项之前和之后, 分别执行以下命令:

```
solve(x^2-a*x+2, x)
solve(x^2+2, x)
solve(x^3=1, x)
```

对于 1 次和 2 次多项式, 系统可以计算出精确根 (针对 3 次和 4 次多项式的卡尔丹公式 (Cardan) 和法拉利公式 (Ferrari) 默认不被使用, 因为求解出的表达形式极其繁琐、难以处理)。对于更高次数的多项式, `solve` 函数会显示一条错误信息并返回一个空列表。

对于三角方程, 系统默认仅返回主解 (主要区间内的解)。若要获取所有解 (通解), 需要启用 `All_trig_sol` 选项。请对比在启用与未启用该选项时执行以下命令的效果:

```
solve(cos(x), x)
solve(cos(x)+sin(x), x)
```

函数也可以用来求解方程组。其中, 第一个参数是方程列表, 第二个参数是变量列表。


```
solve([x^2+y-2,x+y^2-2],[x,y])
```

近似求解函数为 `fsolve`。它在菜单中提供了多种可选算法（菜单路径为 `Calc->Num_solve_eq` 以及 `Calc->Num_solve_syst`）。

其中最著名的是牛顿法（*Algorithme de Newton*），该算法有多种变体。所有这些数值算法的基本原理，都是通过计算一个迭代序列的连续项，使其逐步收敛到方程或方程组的解。为此，通常需要根据具体情况指定一个初始点（起点）或一个搜索区间。

```
fsolve((x^5+2*x+1)=0,x,1,newton_solver)
newton(x^5+2*x+1,x,1.0)
newton(x^5+2*x+1,x,1+i)
newton(x^5+2*x+1,x,-1+i)
```

方程	
<code>solve(eq,x)</code>	方程的精确求解
<code>solve([eq1,eq2],[x,y])</code>	方程组的精确求解
<code>fsolve(eq,x)</code>	方程的近似求解
<code>fsolve([eq1,eq2],[x,y])</code>	方程组的近似求解
<code>newton</code>	牛顿法
<code>linsolve</code>	线性方程组
<code>proot</code>	多项式的近似根

3.5 常微分方程

与前两节类似，微分方程的求解也分为精确求解（并非所有方程都能求出解析解）和数值近似求解。

精确求解：通过 `desolve` 函数实现。

导数表示：未知函数 y 的导数可以直接写为 y' , y'' 等形式，系统内部会自动将其转化为 `diff(y)`, `diff(diff(y))`。

任意常数：如果未指定初始条件，计算结果将以任意常数的形式给出通解。

```
desolve(y'=y,y)
desolve(y''+2*y'+y=0,y)
desolve((x^2-1)*y'+2*y=0,y)
```

初始条件在 `Xcas` 中被视为额外的方程，它们与微分方程本身一同组成一个列表传递给 `desolve` 函数。

```
desolve([y'=y,y(0)=1],y)
desolve([y''+2*y'+y=0,y(0)=1],y)
desolve([y''+2*y'+y=0,y(0)=1,y'(0)=1],y)
desolve([y''+2*y'+y=0,y(0)=1,y(1)=0],y)
desolve([(x^2-1)*y'+2*y=0,y(0)=1],y)
desolve((t^2-1)*diff(y(t),t)+2*y(t)=0,y(t))
```

函数 `odesolve` 允许通过数值方法求解过点 (x_0, y_0) 的常微分方程 $y' = f(x, y)$
例如：

```
odesolve(sin(x*y), [x,y], [0,1], 2)
```

用于计算 $y(2)$ ，其中 $y(x)$ 是满足初始条件 $y(0) = 1$ 的常微分方程 $y'(x) = \sin(xy)$ 的数值解。`plotode` 函数用于绘制微分方程解的图像，`plotfield` 用于绘制切线场（方向场/斜率场）。`interactive_odeplot` 函数则在绘制切线场的同时提供交互功能，允许在图表上点击任意位置，直接生成并绘制经过该点击点的积分曲线（解曲线）。

```
plotfield(sin(x*y), [x,y])
plotode(sin(x*y), [x,y], [0,1])
erase()
interactive_plotode(sin(x*y), [x,y])
```

常微分方程	
<code>desolve</code>	精确求解 / 解析解
<code>odesolve</code>	近似求解 / 数值解
<code>plotode</code>	轨迹 / 解曲线绘制
<code>plotfield</code>	矢量场 / 切线场绘制
<code>interactive_plotode</code>	可点击交互界面

4 代数工具

4.1 整数算术

整数的相关运算存放在菜单 `Cmds->Entier` 中。模 p 算术（同余计算）可以通过 `% p` 来实现。一旦定义了一个模 p 的整数（例如 `a:=3%5`），后续所有涉及该变量的计算都将在同余类环 $\mathbb{Z}/p\mathbb{Z}$ 中自动完成：`a*2` 返回 `1%5`（因为 $6 \equiv 1 \pmod{5}$ ），`1/a` 返回 `2%5`，... 若要高效计算模 p 幂次（快速模幂运算），既可以直接使用模 p 整数求幂，也可以使用 `powermod` 或 `powmod`。

```
a:=3%5
a+12
a^4
powermod(3,4,5)
```

整数	
<code>a%p</code>	$a \bmod p$
<code>powmod(a,n,p)</code>	$a^n \bmod p$ (模幂运算)
<code>irem</code>	欧几里得除法的余数
<code>iquo</code>	欧几里得除法的商
<code>iquorem</code>	商和余数
<code>ifactor</code>	质因数分解
<code>ifactors</code>	质因数列表
<code>idivis</code>	约数列表
<code>gcd</code>	最大公约数
<code>lcm</code>	最小公倍数
<code>iegcd</code>	裴蜀等式
<code>iabcuv</code>	返回系数 $[u,v]$ 且满足 $au + bv = c$
<code>is_prime</code>	判断整数是否为质数
<code>nextprime</code>	下一个质数
<code>previousprime</code>	上一个质数

4.2 多项式与有理分式

多项式处理的相关函数存放在菜单 `Cmds->Polyn\^omes`.

使用 `normal` 或 `expand` 进行展开，或者更常见地用于将分式化简为既约分式（最简分式）。使用 `factor` 进行因式分解。运算结果取决于所选取的数域：默认情况：如果系数是精确数，计算将在有理数域上进行；如果包含浮点数，则在实数域上进行。复数域：若需要在复数域（精确值或近似值）上计算，需要通过状态栏/配置按钮 `Config:...` 激活 `Complexe` 选项。

有限环与有限域：可以将系数声明为模 p 的整数，以便在有限环 $\mathbb{Z}/p\mathbb{Z}$ 中工作（使用 `%` 命令），或使用 `GF` 命令在有限域（Galois Field）中工作。请在激活 `Complexe` 选项之前和之后，分别执行以下命令：

```
P:=x^4-1
factor(P)
gcd(P,x^3-1)
divis(P)
propfrac(x^4/P)
partfrac(4/P)
Q:=(x^4+1)%3
factor(Q)
G:=GF(2,8,['a','G'])
factor(G(a^3)*x^2+1)
genpoly(5,3,x)
genpoly(2,3,x)
genpoly(2*y+5,3,x)
```

多项式	
normal	标准形式（展开并化简）
expand	展开形式
ptayl	泰勒展开形式
peval ou horner	使用霍纳算法（秦九韶算法）在某点求值
genpoly	在指定点代入求值的多项式
canonical_form	二次三项式的配方式（标准型 / 顶点式）
coeff	系数（系数列表）
poly2symb	从代数表达式转换为符号形式（系数列表）
symb2poly	从符号形式（系数列表）转换为代数表达式
pcoeff	由根构成的多项式
degree	次数（最高次数）
lcoeff	最高次项系数（首项系数）
valuation	最低次项的次数
tcoeff	最低次项系数（尾项系数）
factor	因式分解
factors	不可约因式列表
divis	因式（约式）列表
collect	整数域上的因式分解
froot	根及其重数
proot	根的近似值（数值根）
sturmab	区间内根的个数
getNum	有理分式的分子
getDenom	有理分式的分母
propfrac	分离整式部分与真分式部分
partfrac	部分分式分解
quo	多项式除法的商
rem	多项式除法的余式
gcd	最大公因式
lcm	最小公倍式
egcd	多项式的裴蜀等式
divpc	升幂除法（幂级数展开除法）
randpoly	随机多项式
cyclotomic	圆分多项式
lagrange	拉格朗日插值多项式
hermite	埃尔米特多项式
laguerre	拉盖尔多项式
tchebyshev1	第一类切比雪夫多项式
tchebyshev2	第二类切比雪夫多项式

4.3 三角学

菜单 `Cmds->R\>ecl->Transcendental` 包含了三角函数（圆函数）与双曲函数及其各自的反函数。

线性化与展开：使用 `tlin` 进行三角函数的线性化（将降幂、积化和差化为一次项之和），使用 `texpand` 进行三角函数的展开（将和角、倍角展开）。重写与变换：菜单中还提供了许多其他的三角恒等式变换与重写函数（如指数与三角函数互化、半角切公式换元等）。

- Expression->Trigo (三角变换) : 转换为 $\tan(x/2)$ (halftan, 即万能公式换元), 将正切化为正弦和余弦 (tan2sincos), ...
- Expression->Trigo exp (三角与指数互化) : 利用欧拉公式将三角函数转换为指数形式 (trig2exp), 将指数形式转换为三角函数 (exp2trig), 将指数形式转换为幂函数形式 (exp2pow)....
- Expression->Trigo inv (反三角函数变换) : 将指数形式转换为幂函数形式

```
exp2pow(exp(3*ln(x)))
exp2trig(exp(i*x))
trig2exp(cos(x))
E:=sin(x)^4+sin(x)^3
El:=tlin(E)
texpand(E1)
tsimplify(E)
tsimplify(E1)
tsimplify(E-E1)
halftan(E)
trig2exp(E1)
Et:=trigtan(E)
tan2sincos(Et)
tan2sincos2(Et)
tan2cossin2(Et)
```

三角学	
tlin	线性化
tcollect	线性化并合并同类项
texpand	多项式形式
trig2exp	三角函数转换为指数形式
exp2trig	指数形式转换为三角函数
hyp2exp	双曲函数转换为指数形式

4.4 向量与矩阵

向量 (vecteur) : 由数字构成的列表 (如 [1, 2, 3])。

矩阵 (matrice) : 由其行向量构成的列表 (即嵌套列表, 如 [[1, 2], [3, 4]])。

矩阵乘法 (produit matriciel) : 直接使用普通乘号 * 计算。

向量运算法则: 向量在默认情况下均视为行向量 (vecteurs lignes)。当矩阵在右侧乘上一个行向量时, 系统会自动将其当作列向量 (vecteur colonne) 处理。

数量积/点积 (produit scalaire) : 若 $\mathbf{v}$ 和 $\mathbf{w}$ 为两个同维度的向量, 计算 $\mathbf{v} \cdot \mathbf{w}$ 将直接返回它们的点积。

```
A:=[[1,2,3],[4,5,6],[7,8,9]]
v:=[1,1,1]
v*v
A*v
v*A
```

```

B:=[[1,1,1],[2,2,2]]
A*B
B*A
A*tran(B)

```

通过将双下标 (j,k) 映射为实数 $a(j,k)$ 的二元函数, 我们可以使用 `makemat` 或 `matrix` 函数来构造一个矩阵。使用 `makemat` 函数时, 下标从1开始计数, 使用 `matrix` 函数时, 下标从1开始计数。(译注: 实测在计算器软件 `khicas50` 和计算机软件 `xcas` 中, 使用两种函数的两个自变量都从0开始变化; 元素行列表示是从0开始, 即第一行第一列的数值下标表示为[0,0])

```

makemat((j,k)->j+2*k,3,2)
matrix(3,2,(j,k)->j+2*k)

```

还可以使用 `blockmatrix` 命令通过分块的方式将多个子矩阵拼接构建为一个大矩阵。

```

A:=makemat((j,k)->j+2*k,3,2)
B:=idn(3)
blockmatrix(1,2,[A,B])
blockmatrix(2,2,[A,B,B,A])

```

可以通过在方括号内使用逗号隔开的两个下标来访问矩阵的一个元素。第一个下标是行下标, 第二个是列下标。下标从0开始。例如, 若 $A:=\begin{bmatrix} 0 & 2 \\ 1 & 3 \\ 2 & 4 \end{bmatrix}$, 则 `A[2,1]` 返回 4。要提取矩阵的一个子块, 可以使用区间作为下标: `A[1..2,0..1]` 返回由第 1 到 2 行和第 0 到 1 列组成的子块。

请注意, 每次修改一个系数时, `xcas` 的矩阵都会被完整复制。如果在程序中连续修改同一个 (大型) 矩阵的许多系数, 这会导致非常严重的性能开销。

向量与矩阵	
<code>v*w</code>	数量积 (点积)
<code>cross(v,w)</code>	向量积 (叉积)
<code>A*B</code>	矩阵乘法
<code>A.*B</code>	按元素逐项相乘
<code>1/A</code>	逆矩阵
<code>tran</code>	转置矩阵
<code>rank</code>	秩
<code>det</code>	行列式
<code>ker</code>	核 (零空间) 的基
<code>image</code>	像 (列空间) 的基
<code>idn</code>	单位矩阵
<code>ranm</code>	随机系数矩阵

4.5 线性方程组

`linsolve` 函数用于求解线性方程组列表, 语法与 `solve` 相同。也可以使用 `simult` 来求解仅右端项不同的多个线性方程组, 将方程组的系数矩阵作为第

一个参数，将一列（或多列）为右端项的矩阵作为第二个参数；或者对通过在系数矩阵右侧增广右端项得到的矩阵使用 `rref`（如果 `b` 是列矩阵，则使用 `border(A,tran(b))`）。当方程组无解时，`linsolve` 返回空列表，`simult` 返回错误信息，`rref` 返回的矩阵中其中一行全为 `M`（最后一个系数除外）。当方程组有无穷多解时，`linsolve` 返回关于某些变量的解，`simult` 仅返回其中一个解，`rref` 返回有一行或多行全为 `0` 的矩阵。

下面的示例涉及方程组：

$$\begin{cases} x + y + az = 1 \\ x + ay + z = 1 \\ ax + y + z = -2 \end{cases}$$

当 $a \neq 1$ 且 $a \neq -2$ 时它有唯一解，当 $a = 1$ 时它无解，当 $a = -2$ 时它有不定解（有无穷多解）。

```
linsolve([x+y+a*z=1,x+a*y+z=1,x+a*y+z=-2],[x,y,z])
a:=1
linsolve([x+y+a*z=1,x+a*y+z=1,x+a*y+z=-2],[x,y,z])
a:=-2
linsolve([x+y+a*z=1,x+a*y+z=1,x+a*y+z=-2],[x,y,z])
purge(a)
A:=[ [1,1,a],[1,a,1],[a,1,1]]
solve(det(A),a)
A1:=subst(A,a=1)
rank(A1)
image(A1)
ker(A1)
A2:=subst(A,a=-2)
rank(A2)
image(A2)
ker(A2)
b:= [1,1,-2]
B:=tran(b)
simult(A,B)
simult(A1,B)
simult(A2,B)
M:=blockmatrix(1,2,[A,B])
rref(M)
rref(border(A,b))
rref(border(A1,b))
rref(border(A2,b))
```

线性方程组	
<code>linsolve</code>	求解方程组
<code>simult</code>	同时求解多个方程组
<code>rref</code>	高斯-约当(Gauss-Jordan)消元
<code>rank</code>	秩
<code>det</code>	方程组的行列式

4.6 矩阵化简

`jordan` 函数接收一个矩阵 A 作为输入，并返回一个过渡矩阵 P 和若尔当标准形 J ，满足 $P^{-1}AP = J$ 。如果 A 可对角化，则 J 为对角矩阵，其对角线上包含 A 的特征值；如果 A 不可对角化，则 J 在主对角线上方包含 "1" 或 "0"。对于精确矩阵和符号矩阵，仅可求得能通过 `solve` 计算出的特征值。对于近似数值矩阵，系统使用数值算法，但在存在重特征值或彼此极接近的特征值时，算法可能会失败。接下来示例中的矩阵 A 具有二重特征值

1 和 2。当 $a = 0$ 时它是可对角化的，当 $a \neq 0$ 时不可对角化。

```
A:= [[1,1,-1,0],[0,1,0,a],[0,-1,2,0],[1,0,1,2]]
```

```
factor(poly2symb(simplify(pcar(A))))
```

```
jordan(A)
```

```
eigenvals(A)
```

```
eigenvects(A)
```

```
jordan(subs(A,a=0))
```

```
eigenvects(subs(A,a=1))
```

```
jordan(evalf(subs(A,a=0)))
```

```
jordan(evalf(subs(A,a=1)))
```

一些由幂级数定义的函数，只要已知如何计算其若尔当标准形，就可以推广到矩阵上。其中最有用的是指数函数。

```
A:= [[0,1,0],[0,0,1],[-2,1,2]]
```

```
jordan(A)
```

```
exp(A)
```

```
ln(A)
```

```
sin(A)
```

矩阵化简	
<code>jordan</code>	对角化或若尔当化化简
<code>pcar</code>	特征多项式的系数
<code>pmin</code>	最小多项式的系数
<code>eigenvals</code>	特征值
<code>eigenvects</code>	特征向量

5 图形表示

要在 Xcas 中获得图形表示，需要输入一个输出结果为图形对象的命令。为了帮助您进行最常见的图形表示，**Graphic** 菜单为您提供了对话框，用于自动生成命令行并执行它以显示所需的图形。

如果您想在同一个视图表示多个图形对象，必须用分号；将创建它们的命令隔开。您也可以创建一个图形 (**Geo** 菜单, **Nouvelle figure**/新图形) 并使用 **Geo** 菜单来创建几何对象。

每个图形命令都会创建一个二维或三维图形对象，并在 Xcas 窗口中作为响应输出为一张图像。在这张图像的右侧，放大(in)和缩小(out)缩放按钮可以放大或缩小视图，箭头按钮则可以平移视图。

默认参数（特别是横坐标和纵坐标的显示区间）可以在从菜单 **cfg->Configuration graphique** (配置->图形配置) 进入的图形配置窗口中进行修改。

最后请注意，二维图形对象也会显示在一个名为 **DispG** (**Display Graphics**/显示图形) 的窗口中，您可以通过菜单 **Cfg->Montrer->DispG** 或使用 **DispG.** 命令使其显示。区别在于，接连生成的图形会在各自的响应窗口中单独绘制，而在 **DispG** 窗口中它们则是相互叠加显示的。您可以通过 **ClrGraph** 命令清除 **DispG** 窗口。

5.1 曲线绘图

要快速创建简单曲线的图像，建议使用 **Graphic** 菜单。

用于绘制函数代表性曲线的指令是 **plot**，其参数为一个表达式或希望绘制其图形的表达式列表，接着是变量（可以指定变量的取值区间）。为了区分多条曲线，可以使用第三个参数，例如 **couleur=** 后面跟上要使用的颜色列表。颜色可以通过其法语名称、英语名称或编号来进行编码。**couleur** 函数会改变后续所有图形函数的基准颜色。**tangent** 函数用于获取曲线在某一点处的切线。

```
E:=(2*x+1)/(x^2+1);plot(E)
plot(E,x=-2..2,color=red)
couleur(vert);plot(E,couleur=rouge);tangent(plot(E),0)
DispG()
plot([sin(x),x,x-x^3/6],x=-2..2,couleur=[rouge,bleu,vert])
erase
li:=[(x+k*0.5)^2$(k=-5..5)];
plot(li,x=-8..8,couleur=[k$(k=0..10)])
```

`plotparam` 函数用于绘制 $(x(t), y(t))$ 。需要将这两个坐标定义为一个复数表达式，其中 $x(t)$ 为实部， $y(t)$ 为虚部。`plotpolar` 函数用于绘制极坐标下的曲线。命令 `plotimplicit(f(x,y), x, y)` 用于绘制 $f(x,y) = 0$ 解集的图形。

```
plotparam(sin(t)^3+i*cos(t)^3,t,0,2*pi)
plotparam(t^2+i*t^3,t,-1,1)
plotpolar(1/(1-2sin(t/2)),t,0,4*pi)
plotpolar(tan(t)+tan(t/2),t,0,2*pi)
plotimplicit(x^2+4*y^2-4,x,y)
```

曲线绘图	
<code>plot</code>	单变量表达式的图像
<code>plotfunc</code>	单变量或双变量表达式的图像
<code>couleur</code>	选择绘制曲线的颜色
<code>tangent</code>	曲线的切线
<code>plotparam</code>	参数方程曲线
<code>plotpolar</code>	极坐标曲线
<code>plotimplicit</code>	隐函数曲线

5.2 二维图形对象

由于 `Xcas` 也是一个平面几何软件，因此许多图形（在通过 `Alt+g` 调出的屏幕中）可以通过 `Geo` 菜单的命令来绘制，例如多边形、圆锥曲线.....这些命令的参数通常是点（`point` 命令），一般可以直接通过其复数坐标（复数形式）输入。例如 `cercle(2+3*i,2)` 绘制以点 $(2,3)$ 为圆心、半径为 2 的圆。`legende` 命令允许在特定位置放置一段文本，该位置同样由一个复数指定。`polygonplot` 和 `scatterplot` 函数接收一个横坐标列表和一个纵坐标列表作为输入。

```
lx:= [k$ (k=1..10)]
ly:=apply(sin,lx)
polygonplot(lx,ly)
erase
scatterplot(lx,ly)
polygone_ouvert(lx+i*ly)
```

二维图形对象	
legend	从给定点开始添加文本
point	由坐标复数或两个坐标给出的点
segment	由两点给出的线段
droite	由方程或两点给出的直线
cercle	由圆心和半径给出的圆
inter	曲线的交点
equation	笛卡尔方程
parameq	参数方程
polygonplot	折线
scatterplot	散点图
polygone	闭合多边形
polygone_ouvert	开放多边形

5.3 三维图形对象

- 要绘制由方程 $z = f(x,y)$ 定义的曲面，可以使用 `plotfunc` 命令，其参数为曲面方程以及两个变量的列表。也可以指定变量的取值范围和离散化程度（网格密度）。

```
plotfunc(x^2-y^2, [x,y])
plotfunc(x+y^2, [x=-5..5,y=-2..2], xstep=0.5, ystep=0.1,
affichage=vert+rempli)
```

可以得到一个三维空间中的曲面。要修改视角，可以在可视化立方体外部使用鼠标，或者在三维图形内部点击，然后使用 `x,X,y,Y,z,Z` 键（绕各轴旋转）、`+` 和 `-` `pour les zooms` 键进行缩放，以及方向箭头键和 `page up/down` 键来更改视角窗口（如果在 `plotfunc` 的参数中未指定，默认计算窗口由图形配置确定）。

- 还可以使用 `plotparam` 绘制参数曲面，其第一个参数是包含该点三个坐标的列表，接下来的两个参数是两个参数变量：

```
plotparam([u,v,u+v], u=-1..1, v=-2..2)
```

- 要绘制空间中的参数曲线，同样使用 `plotparam` 命令，但只使用一个参数：

```
plotparam([u,u^2,u^3], u=-1..1)
```

- 也可以在通过 `Alt+h` 调出的三维图形中绘制三维几何对象，然后使用 `Geo`

菜单的命令，例如：`point`（点），`droite`（直线），`plan`（平面），`polygone`（多边形），`sphere`（球体）...

```
plan(z=x+y);
droite(x=y, z=y);
A:=point(1,2,3); B:=point(2,-1,1); C:=point(1,0,0);
plan(A,B,C,couleur=cyan);
droite(A,B,affichage=line_width_3)
```

三维图形对象	
plotfunc	由方程定义的曲面
plotparam	参数曲面或参数曲线
point	由 3 个坐标给出的点
droite	由 2 个方程或两点给出的直线
plan	由 1 个方程或 3 个点给出的平面
sphere	由球心和半径给出的球体
cone	由顶点、轴线方向和张角给出的圆锥
inter	交集（交点 / 交线 / 交集图形）
polygone	多边形（闭合）
polygone_ouvert	开放多边形（折线）

6 编程

6.1 编程语言

Xcas 允许像任何其他编程语言一样编写程序。以下是它的主要特点。

- 这是一门函数式语言（Functional Language）。函数的参数可以是另一个函数。在这种情况下，既可以在命令中直接传入作为参数的函数名称，也可以直接传入其匿名定义（Lambda 表达式）：例如 `function_diff(f)` 或 `function_diff(x->x^2)`。
- 程序（Program）与函数（Function）之间没有区别：函数会返回最后一条被评估指令的值，或者返回关键字 `return` 之后的内容。与所有计算环境一样，编写程序本质上就是通过添加所需的新函数来扩展 Xcas 的功能。结构化编程则体现在相互调用的不同函数之间的层级组织上。
- 该语言是无类型（动态类型）语言。仅区分未声明的全局变量（Global Variables）与在函数开头显式声明的局部变量（Local Variables）。

在程序中，当调用一个未被赋值为列表（list）、序列（sequence）或矩阵（matrix）的带下标变量时，系统创建的是一个表/哈希表（table），而不是列表。表是一种类似于列表和序列的容器，区别在于它的索引可以是整数以外的其他类型（例如字符串等）。如果 `a` 是一个未赋值的符号变量，执行命令 `a[4]:=2` 会创建一个表 `a`。若要使 `a` 成为一个列表，必须先将其初始化为列表，例如 `a:=[0$10]`（如果已知列表长度）或 `a:=[]`，然后再执行 `a[4]:=2`。因此，尽管该语言是无类型的，仍强烈建议在使用变量之前对其进行初始化。

函数的声明语法如下：

```
nom_fonction(var1,var2,...):={
local var_loc1, var_loc2,... ;
  instruction1;
  instruction2;
  ...
}
```

语法既可以使用法语关键字，也可以采用 C++ 语言的语法。

法语指令	
赋值	a:=2;
表达式输入	saisir("a=",a);
字符串输入	saisir_chaine("a=",a);
输出（打印）	afficher("a=",a);
返回值	retourne(a);
循环中断(跳出循环)	break;
条件分支(选择结构)	si <condition> alors <inst> fsi; si <condition> alors <inst1> sinon <inst2> fsi;
计步循环	pour j de a jusque b faire <inst> fpour; pour j de a jusque b pas p faire <inst> fpour;
直到型循环	repetier <inst> jusqua <condition>;
当型循环	tantque <condition> faire <inst> ftantque;
执行型/后条件循环	faire <inst1> si <condition> break;<inst2> ffaire;

类 C++ 风格指令	
赋值	a:=2;
表达式输入	input("a=",a);
字符串输入	textinput("a=",a);
输出(控制台输出)	print("a=",a);
返回值	return(a);
循环中断(跳出循环)	break;
条件分支(选择结构)	if (<condition>) {<inst>; if (<condition>) {<inst1>} else {<inst2>;
计步循环	for (j:= a;j<=b;j++) {<inst>; for (j:= a;j<=b;j:=j+p) {<inst>;
直到型循环	repeat <inst> until <condition>;
当型循环	while (<condition>) {<inst>;
执行型循环	do <inst1> if (<condition>) break;<inst2> od;

对于条件测试（判断语句），条件是一个布尔值（Boolean），它是使用常用运算符得到的逻辑表达式的结果。

逻辑与关系运算符			
==	测试相等	!=	测试不相等
<	测试严格小于	>	测试严格大于
<=	测试小于或等于	>=	测试大于或等于
&&, et	中缀布尔与运算符	, ou	中缀布尔或运算符
vrai	布尔值真（true 或 1）	faux	布尔值假（false 或 0）
non, !	逻辑非/取反		

注意, i 表示虚数单位 $\sqrt{-1}$, 不能用作循环变量。

`break`; 指令允许跳出 (终止) 当前循环, `continue`; 指令允许立即跳过当前循环体剩余部分并直接进入下一次迭代。Xcas 可以识别许多语法变体, 特别是在与 Maple、MuPAD 和 TI89/Voyage 200 的兼容模式下。可以通过

```
try {bloc_erreurs_capturees}
catch (variable)
    {bloc_execute_si_erreur}
```

结构来捕获运行时错误 (异常处理)。

例如:

```
try{A:=idn(2)*idn(3)}
catch(erreur)
{print("l'erreur est "+erreur)}
```

6.2 一些示例

要编写程序, 建议通过菜单 `Prg->Nouveau programme` (程序 -> 新建程序) 打开程序编辑器。编辑器的 `Prg` 菜单可以方便地插入各种编程结构。随后可以独立于当前工作会话 (Session) 保存程序文本, 以便日后在其他会话中直接调入使用。

下面是一个计算两个整数欧几里得除法 (带余除法) 商和余数的程序, 其中使用了返回商的 `iquo` 函数和返回余数的 `irem` 函数 (在 Xcas 中也可直接使用内置的 `iquorem` 函数)。

输入校验/错误处理功能的带余除法 (欧几里得除法) 函数

idiv2	
<pre>idiv2(a,b):={ local q,r; if (b!=0) { q:=iquo(a,b); r:=irem(a,b);} else { q:=0; r:=a; } return [q,r]; }</pre>	<pre>idiv2(a,b):={ local q,r; si b!=0 alors q:=iquo(a,b); r:=irem(a,b); sinon q:=0; r:=a; fsi retourne [q,r]; }</pre>

在程序编辑器中输入这个 `idiv2` 函数, 点击 `OK` 按钮进行测试, 然后将其保存 (例如命名为 `idiv2.cxx`)。您可以在命令行中直接调用该函数, 例如输入 `idiv2(25,15)`。日后在其他的 Xcas 会话 (Session) 中, 只需执行 `read("idiv2.cxx")` 命令导入脚本, 或者直接在程序编辑器中打开该文件并点击 `OK` 确认, 即可再次使用该函数。

下面展示计算两个整数最大公约数 (PGCD / GCD) 的两种实现版本: 迭代版本 (Version itérative) 与递归版本 (Version récursive)。

pgcd_迭代版本	
<pre>pgcdi(a,b):={ local r; while (b!=0) { r:=irem(a,b); a:=b; b:=r; } return a; }::</pre>	<pre>pgcdi(a,b):={ local r; tantque b!=0 faire r:=irem(a,b); a:=b; b:=r; ftantque retourne a; }::</pre>

pgcd_递归版本	
<pre>pgcdr(a,b):={ if (b!=0) return a; return pgcdr(b,irem(a,b)); }::</pre>	<pre>pgcdr(a,b):={ si b!=0 alors retourne a;fssi retourne pgcdr(b,irem(a,b)); }::</pre>

有时程序可能无法一试即成、按照预想的方式运行 (!)。此时可以使用 `debug` 命令在单步模式 (mode pas-à-pas) 下运行程序以进行调试与排错。更多细节可参阅菜单 `Aide->Interface` (帮助 -> 界面)。例如, 对于 `idiv2` 程序, 可以通过输入以下命令启动调试:

```
debug(idiv2(25,15))
```

在通过 `sst` (Single Step, 单步运行) 按钮逐条指令执行时, 调试器会自动显示参数 `a, b` 以及局部变量 `q, r` 的实时数值状态。

6.3 编程风格

`Xcas` 是解释型语言而非编译型语言。相比于程序行数, 真正影响计算效率与运算时间的是实际执行的指令数量。通常来说, 在 `CAS` (计算机代数系统) 中, 使用列表 (List) 或序列 (Sequence) 配合高阶内置函数, 比直接编写显式循环 (Loop) 效率更高、速度更快。以下是计算 `5000!` 的几种不同编写方式, 您可以对比它们的执行时间:

```
5000!
product([n$(n=1..5000)])
product(cumSum([1$5000]))
f:=1; (f:=f*n)$ (n=2..5000)::f
f:=1; for(n:=1;n<=5000;n++) {f:=f*n}
f:=1;n:=1; while(n<5000) {n:=n+1; f:=f*n}
f:=1; (f:=f*n)$ (n=2..5000)
```

执行速度有时与程序的清晰度 (可读性) 相矛盾, 因此我们必须做出权衡折衷。在日常使用中, 计算时间往往不是核心瓶颈: 我们通常使用像 `Xcas` 这样的解释型语言来测试算法逻辑和搭建原型 (Maquettes)。而实际生产环境中的大规模工程应用, 则会采用像 `C++` 这样的编译型语言进行编写 (例如调用 `giac` 符号计算库来实现计算机代数系统功能)。

7 附有 Xcas 解答的练习题

7.1 函数及其图像表示：高中毕业班级别

7.1.1 练习 1

设定义在 $\mathbb{R}-\{3\}$ 上的函数 f 解析式为：

$$f(x) = (x+1)\ln|x-3|$$

1. 求一阶导数 f' 与二阶导数 f'' ，并推导 f' 的单调性。
2. 计算 f' 在 $-\infty$ 及 3 (左极限) 处的极限。
3. 证明 f' 在 $]-\infty; 3[$ 上有且仅有一个零点 α ，并确定区间范围。
4. 分析 $f'(x)$ 在 $\mathbb{R}-\{3\}$ 上的符号，推导 f 的单调性。
5. 在正交直角坐标系 (单位长度为 1 cm) 中绘制函数 f 的曲线 C 。
6. 计算由曲线 C 、 x 轴以及直线 $x = -1$ 和 $x = 2$ 所围成区域的面积 (单位: cm^2)。

解答

1. 输入以下指令定义函数 f ：

```
f(x):=(x+1)*ln(abs(x-3))
```

输入以下指令定义导函数 f' ：

```
f1:=function_diff(f);
```

接着，

```
f1(x)
```

得到：

```
ln(abs(x-3))+(x+1)/(x-3)
```

因此 $f'(x) = \ln|x-3| + \frac{x+1}{x-3}$ 。
输入以下指令以定义函数 f'' ：

```
f2:=function_diff(f1);
```

接着，

```
f2(x)
```

得到：

```
1/(x-3)+1/(x-3)+(x+1)*(-(1/((x-3)^2)))
```

接着，为了简化书写，输入：

```
normal(f2(x))
```

得到：

$$(x-7)/(x^2-6x+9)$$

或者为了因式分解，输入：

```
factor(f2(x))
```

得到：

$$(x-7)/((x-3)^2)$$

因此 $f''(x) = \frac{x-7}{(x-3)^2}$

另一种方法

输入以下指令以定义函数 f ：

```
f(x):=(x+1)*ln(abs(x-3))
```

输入以下指令以计算 $f'(x)$ ：

```
dfx:=diff(f(x))
```

得到：

$$\ln(\text{abs}(x-3)) + (x+1)/(x-3)$$

因此 $f'(x) = \ln(|x-3|) + \frac{x+1}{x-3}$
并且，输入以下指令以根据 dfx 定义函数 f' ：

```
f1:=unapply(dfx,x);
```

输入以下指令以计算 $f''(x)$ ：

```
ddfx:=diff(dfx)
```

得到：

$$1/(x-3) + 1/(x-3) + (x+1)*(-1/((x-3)^2))$$

或者为了直接获得因式分解表达式，输入：

```
ddfx:=factor(diff(dfx))
```

得到：

$$(x-7)/((x-3)^2)$$

因此 $f''(x) = \frac{x-7}{(x-3)^2}$

并且，输入以下指令以根据 $ddfx$ 定义函数 f'' ：

```
f2:=unapply(ddfx,x);
```

这种做法的优点在于，可以基于化简或因式分解后的表达式来定义函数 $f_2 = f''$ 。

注意!!! 例如，不能这样写：

$g(x):=\text{normal}(\text{diff}(f(x)))$ 来定义函数 $g = f'$ ，而必须写成 $g:=\text{unapply}(\text{normal}(\text{diff}(f(x))),x)$ ，因为否则求导变量 x 与函数 g 的自变量 x 之间会产生混淆。

2. 输入以下指令以求得 f' 在 $-\infty$ 处的极限:

```
limit(f1(x),x,-infinity)
```

得到:

```
+infinity
```

输入以下指令以求得 f' 在 3^- 处的极限:

```
limit(f1(x),x,3,-1)
```

得到:

```
-infinity
```

3. 因为在 $]-\infty;3[$ 上 $f''(x) < 0$ 所以 $f''(x) < 0$, 所以 f' 在 $]-\infty;3[$ 上连续且从 $+\infty$ 递减至 $-\infty$ 。因此, 在 $]-\infty;3[$ 中存在唯一的 α 使得 $f'(\alpha) = 0$. 输入以下指令以获得 α 的近似值:

```
assume(x<3);fsolve(f1(x),x)
```

得到: $x, 0.776592890991$

接着, 输入以下指令以移除对 x 的假设:

```
purge(x)
```

输入:

```
f1(0.7)
```

得到:

```
0.0937786881525
```

输入:

```
f1(0.8)
```

得到:

```
-0.0297244578175
```

我们有 $f1(0.7) > 0$ 且 $f1(0.8) < 0$, 因此 $0.7 < \alpha < 0.8$.

4. 因为 $f''(7) = 0$, 输入以下指令以求得 f' 在 $]3;+\infty[$ 上的最小值:

```
f1(7)
```

得到:

```
ln(4)+2
```

因此, f' 在 $]3;+\infty[$ 上的最小值为正。

所以, 若 $x \in]-\infty; \alpha[\cup]3;+\infty[$, 则 $f'(x) > 0$; 若 $x \in]\alpha;3[$ 则 $f'(x) < 0$ 。因此, f 在 $]-\infty; \alpha[\cup]3;+\infty[$ 上递增, 在 $]\alpha;3[$ 上递减。

5. 我们求 f 在 $-\infty, +\infty$, 以及 3 处的极限。

输入: